

The Editorial Engine

User Guide and Technical Walkthrough

Project Purpose: Live trend-driven, fact-checked AI content generation & optimization

Technology Stack: Next.js (App Router), TypeScript, Gemini, Supabase, Vanilla CSS

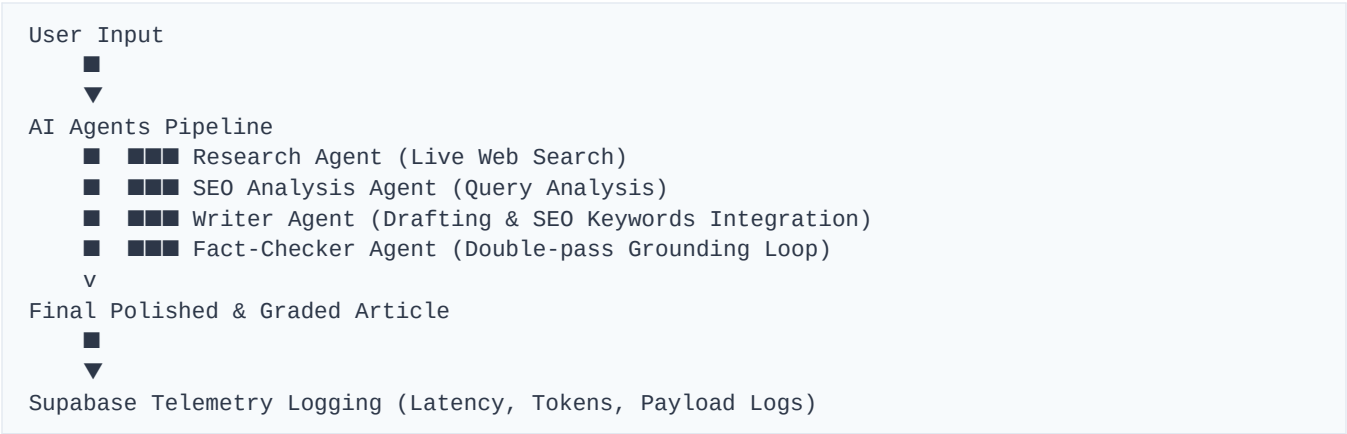
Date/Version: June 2026 / Version 1.0.0

Section 1: Overview

The Editorial Engine solves a critical challenge for modern content creators, marketers, and SEO specialists: **creating high-quality, fact-checked, and search-optimized content efficiently.**

Many automated writing systems generate generic, unverified articles that fail to rank on search engines or provide genuine value to readers. This platform addresses these shortcomings by employing a collaborative network of specialized AI agents. The agents research topics using real-time search grounding, analyze live Google Trends data, cross-examine claims to prevent hallucination, polish style, and grade drafts against custom quality rubrics.

High-Level Pipeline Flow



Section 2: Application Features

Feature A: Article Generation

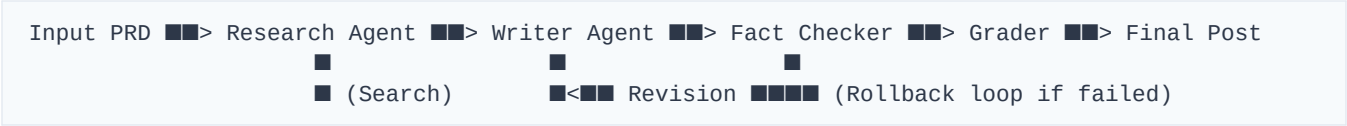
Purpose:

Enables users to draft comprehensive, high-quality, and fact-checked blog articles or documents from scratch based on a Product Requirement Document (PRD), target audience description, or website/business context.

User Steps:

1. Navigate to the **Generate Article** page (/generate).
2. Input your Product Requirement Document (PRD) or basic guidelines in the input text area.
3. (Optional) Pre-fill SEO parameters automatically by using the 'Use in Blog Generator' link from the SEO page.
4. Click **Start Content Generation Pipeline**.
5. Watch the live lifecycle progression, retry loops, and agent logs on the timeline dashboard.
6. Review the final article, citation sources, and quality grader reports.
7. Click the **Export to PDF** button to save a copy.

Workflow:



[Insert Screenshot: Generate Page]

Feature B: SEO Intelligence

Purpose:

Retrieves real-time Google search trend data for a given topic and uses a dedicated SEO agent to suggest primary/secondary keywords, blog titles, and content strategies.

User Steps:

- 1. Navigate to the **SEO Intelligence** page (/seo).
- 2. Enter a **Target Keyword/Topic** (e.g. 'Pokemon cards') and optional business context.
- 3. Click **Analyze SEO Opportunities** to trigger real-time Google Search trend retrieval.
- 4. Analyze the generated recommendations, including related/rising queries and optimized titles.
- 5. Click **Use in Blog Generator** to cache recommendations in local storage and pre-fill the writer form.

Example:

Input Keyword: Pokemon cards

Output Trends & Recommendations:

- **Primary Keyword:** Pokemon card investing
- **Secondary Keywords:** rare Pokemon cards, Pokemon card prices, Pokemon card market trends
- **Suggested Titles:** 'Best Pokemon Cards To Invest In 2026', 'How To Identify Valuable Pokemon Cards'
- **Content Strategy:** Focus on rarity markers, grading tutorials, and pricing fluctuations.

Industry Versatility:

This feature works across diverse sectors. For example: **Trading Cards** (trends on set releases), **Restaurants** (local culinary spikes), **SaaS** (rising integration needs), or **Ecommerce** (seasonal shopping queries).

[Insert Screenshot: SEO Analysis Results]

Feature C: Existing Article Optimizer

Purpose:

Benchmarks an existing draft against current search trends to identify missing keywords, content gaps, and structural improvements.

User Steps:

1. Navigate to the **Existing Article Optimizer** (/optimize).
2. Provide the **Target Keyword**, **Website Context**, and paste the draft content.
3. Click **Analyze & Optimize Article**.
4. Review the SEO score, title evaluation, content gaps list, and readability feedback.
5. Click **Improve Article with AI** to send the draft and optimizer report to the Writer Agent pipeline for a full rewrite.

Example Before/After:

Before (Weak Content): Title: 'Brewing Coffee' (No headers, missing keyword terms like 'v60 technique').

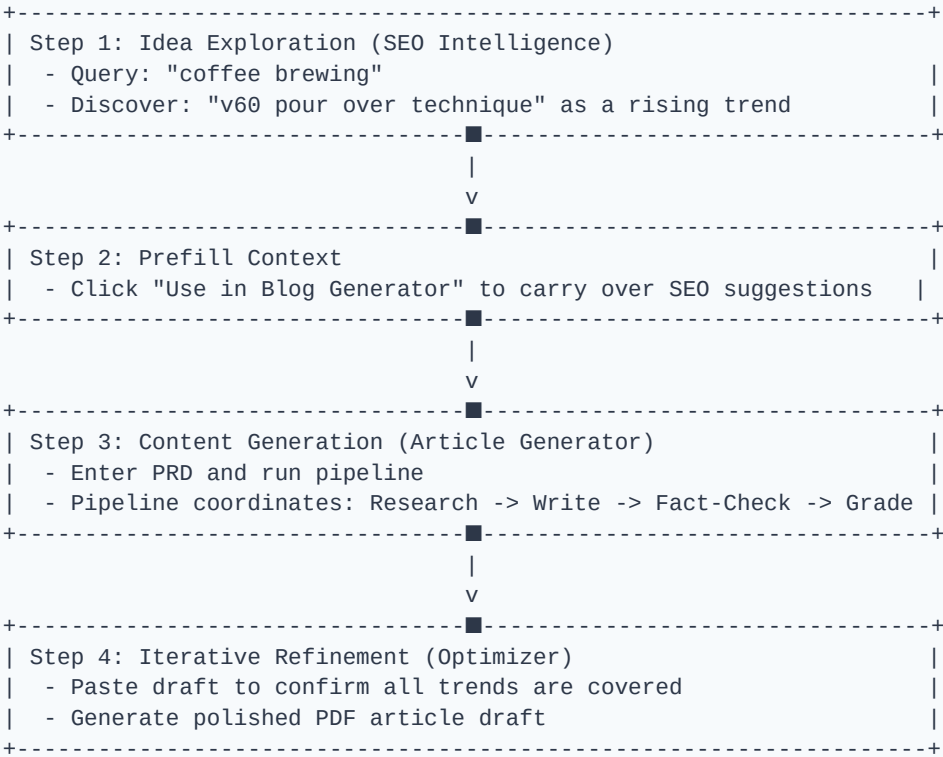
After Report Output:

- **SEO Score:** 52/100
- **Missing Keywords:** 'v60 pour over technique', 'home coffee grinder', 'water-to-coffee ratio'
- **Recommended Titles:** 'Mastering the V60 Pour Over Technique at Home', 'Ultimate V60 Coffee Brewing Guide'
- **Actionable Suggestion:** Add a dedicated heading on choosing grind sizes and coffee-to-water ratios.

[Insert Screenshot: Optimization Results]

Section 3: Complete User Workflow Examples

A typical end-to-end user workflow combines all three features to discover keyword opportunities, generate a fact-checked draft, and audit it for complete search alignment.



Section 4: Developer Setup Guide

Follow these instructions to run the application locally on a development machine.

Prerequisites:

- Node.js: v20.x or later (v20.9.0 recommended)
- npm: v10.x or later (v10.1.0 recommended)
- Active Google Gemini API account
- Supabase project database

Installation Steps:

1. Clone the codebase repository: `git clone <repository-url>`
2. Install npm packages: `npm install`
3. Create the environment file `.env.local` in the root directory: `touch .env.local`
4. Add required keys to `.env.local` (see variables below).
5. Initialize the Supabase database table using `supabase/schema.sql`.
6. Start the Next.js development server: `npm run dev`
7. Open browser at: `http://localhost:3000`

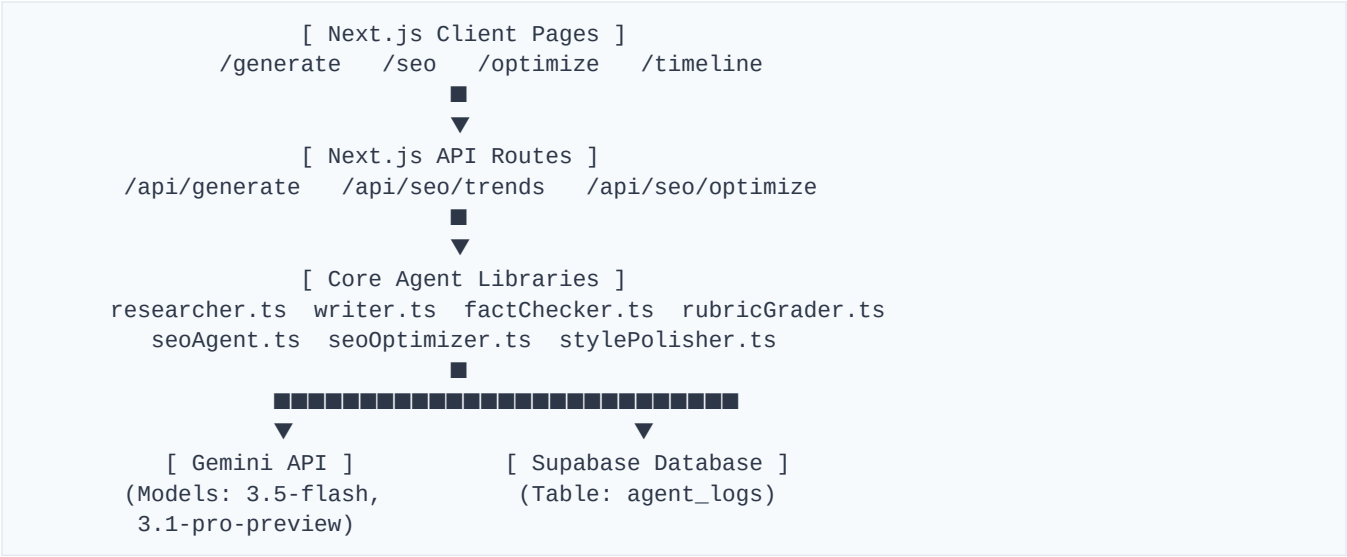
Required Environment Variables (in `.env.local`):

```
# Gemini API Access
GEMINI_API_KEY=your_gemini_api_key_here

# Supabase Database Telemetry Credentials
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_anon_public_key
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
```

Section 5: Project Architecture

The project is structured as a standard Next.js 16 application using App Router, with client pages, serverless API routes, core agent libraries, and database logging via Supabase.



Tech Stack Details:

- **Frontend:** React components styled with Next.js variables and custom dark-mode theme CSS.
- **AI Agents:** Unified @google/genai client calling model endpoints dynamically.
- **Telemetry:** Supabase table tracking execution latency, tokens, inputs, and outputs.

Section 6: AI Agent Overview

The platform is powered by seven distinct agents, each with a defined role in the pipeline.

Agent Name	Model Used	Input	Output	Purpose
Research Agent	gemini-3.5-flash	PRD text	Research summary + sources	Performs live web searches with grounding to verify facts.
Writer Agent	gemini-3.5-flash	PRD + research + SEO keywords	Structured article draft JSON	Drafts the initial post or applies iterative feedback revisions.
Fact-Checker Agent	gemini-3.1-pro-previous	PRD + research + current draft	Passed boolean + unsupported claims list + feedback	Cross-checks draft claims, triggering a revision rollback loop on errors.
Style Polisher Agent	gemini-3.5-flash	Verified draft	Polished draft JSON	Improves vocabulary, flow, and headings without altering facts.
Rubric Grader Agent	gemini-3.1-pro-previous	PRD + final draft + fact-check results	Scores (Clarity, Accuracy, Completeness) + text feedback	Computes objective grades (1-5 scale) and qualitative feedback.
SEO Agent	gemini-3.5-flash	Keyword + context + trends data	Primary/secondary keywords + title options + strategy	Creates search strategy based on real-time trend interests.
SEO Optimizer Agent	gemini-3.5-flash	Draft + keyword + trends	SEO score + gaps + headings + suggestions	Compares draft text against keyword trends and suggests improvements.

Section 7: Troubleshooting

Issue 1: Application fails to boot or compilation errors:

Fix: Ensure Node.js version is 20.x or later. Run `rm -rf .next node_modules package-lock.json && npm install` to refresh modules.

Issue 2: API key warning in logs:

Fix: Check that `.env.local` is placed in the project root directory and the key `GEMINI_API_KEY` has a valid value.

Issue 3: Supabase connection fails or database queries hang:

Fix: Verify public and secret keys in `.env.local`. Make sure the database schema was created in your project using the `supabase/schema.sql` script.

Issue 4: Gemini requests fail with a 500 status:

Fix: Verify your internet connection. Check the Gemini API status dashboard for general outages. If rate limits are exceeded, introduce pauses between executions.

Section 8: Future Improvements

The Editorial Engine is designed with modularity to easily integrate future capabilities:

- **Google Search Console Integration:** Fetch real impression and click data directly to target and refine content gaps automatically.
- **Competitor Monitoring:** Build scraper agents to scan competitor pages, identifying keyword gaps and recommending optimization angles.
- **Automated Content Calendars:** Let the system create monthly content calendars and trigger article drafts on a regular cron schedule.
- **Historical SEO Tracking:** Save search interest history to plot target keyword trends in interactive dashboard charts.
- **Social Media Distribution Hooks:** Generate promotional social copy (Twitter, LinkedIn) corresponding to new articles.